TLS, HTTP/2 & QUIC Security Lab

Handshake Analysis, Certificate Trust and TCP Reset Testing

Valentin Ochioiu
Network Security Student

Folkuniversitetet, Gothenburg
May 2026

Project Overview

This lab analyses how encrypted web protocols establish trust and session keys, how HTTP/2 and HTTP/3 negotiate transport, and how TCP-based sessions react to forged reset packets.

Protocol analysis

- Build a TLS 1.2 HTTPS service with Nginx and OpenSSL.
- Inspect ClientHello, ServerHello, certificate and ECDHE exchange in Wireshark.
- Enable HTTP/2 through ALPN and compare the TLS 1.3 handshake.
- Build HTTP/3 with Caddy and inspect QUIC Initial and Handshake packets.

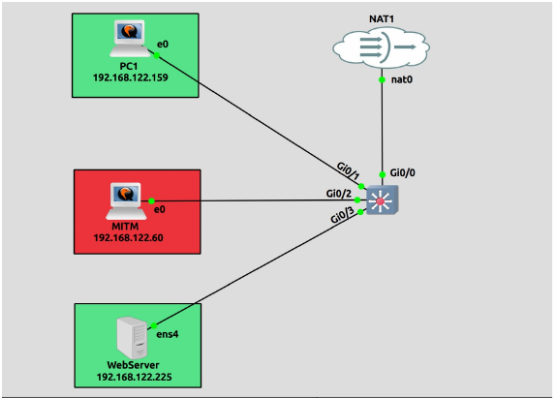
Security focus

Certificate trust, Perfect Forward Secrecy, protected application data and transport-layer disruption.

Validation objective

Compare what changes between TCP/TLS, HTTP/2 and QUIC while keeping the observations grounded in packet captures.

Test Model



GNS3 test topology: client 192.168.122.159, MITM node 192.168.122.60 and web server 192.168.122.225.

Services and tools

- Nginx + OpenSSL for HTTPS and TLS 1.2.
- Caddy v2 for HTTP/3 over QUIC.
- Wireshark for handshake and packet analysis.
- curl for HTTP/1.1, HTTP/2 and HTTP/3 validation.
- tcpkill, hping3 and Scapy for TCP reset experiments.
- ARP spoofing and tcpdump/tshark for the MITM phase.

Scope

The lab is isolated and uses designated test hosts only.

HTTPS Server and Self-Signed Certificate

2.1 - Nginx & OpenSSL

```
debian@server:~$ sudo apt install -y nginx openssl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nginx is already the newest version (1.22.1-9+deb12u7).
openssl is already the newest version (3.0.20-1~deb12u1).
```

2.2 - Självsignerat certifikat

```
debian@server:~$ sudo openssl req -x509 -newkey rsa:4096 -sha256 -days 365 \
-nodes \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/C=SE/O=KTH/OU=NA/CN=demo.local"
...+...........+..+.........+.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++*..+.......+....+.....+....+..+....+...+.....+..+....+...+...........+.....+.+..+...+...+....+.
...........+...........+...+....+...+...+.....+...++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++*.....+...+...+.......+..+......+......+...+..........+..+..............+...+..+..
.+....+......+..............+.....+.......+.........++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++
....+....+...+..+.......+...+........+..........+.....+.........+..+.+.....+......++++++++++++++++
+++++++++++++++++++++++++++++++++++++++++++++*..+.....+.+.....+.......+..+...+..........+..+....+
.....+.+........+.+.....+......+.+..........................+..++++++++++++++++++++++++++++++++++
+++++++++++++++++++++++++++++*.+....+......+...+.....+.......+...............+.....+............+..
......+.........+.+........+....+...+.....+.............+...................+...+..+...+......+..
....+.....+....+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
debian@server:~$ _
```

OpenSSL certificate generation with an RSA 4096-bit private key and SHA-256 signature.

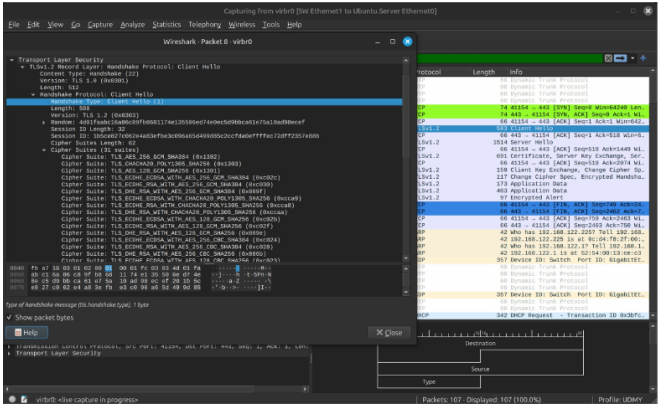
```
  GNU nano 7.2              /etc/nginx/sites-available/default *

# Virtual Host configuration for example.com
#
# You can move that to a different file under sites-available/ and symlink that
# to sites-enabled/ to enable it.
#
server {
    listen 443 ssl;
    server_name demo.local;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    ssl_protocols TLSv1.2;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/html;
}
```

Nginx configured for HTTPS on port 443 with TLS 1.2.

Trust model

The certificate follows X.509 structure, but Subject and Issuer are the same because it is self-signed. Browsers therefore cannot build a chain to a trusted CA.

TLS 1.2: ClientHello



Wireshark view of the ClientHello and the advertised cipher suites/extensions.

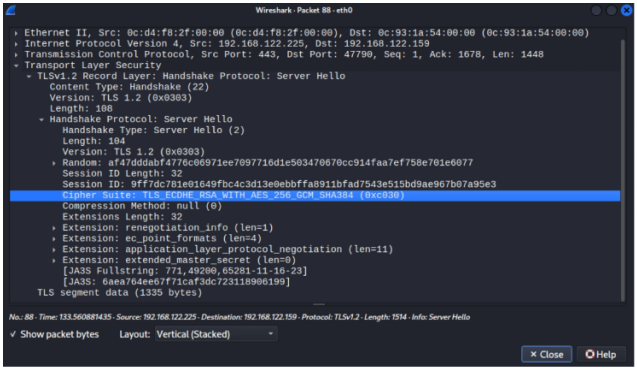
What the client advertises

- Supported TLS versions.
- Cipher suites and cryptographic capabilities.
- Extensions used to negotiate session features.
- A new TLS session is created for each independent curl connection.

Observed result

The server later selected TLS 1.2 because of the Nginx configuration.

TLS 1.2: Selected Cipher Suite

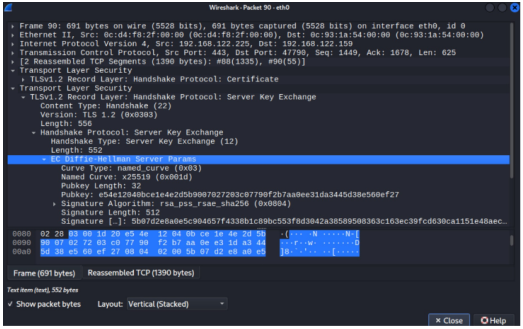


ServerHello selecting TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384.

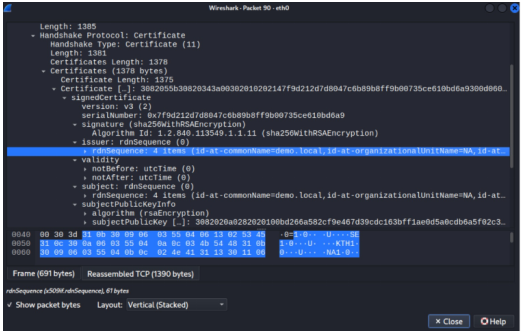
Cipher components

- **ECDHE** — ephemeral key agreement.
- **RSA** — server authentication through the certificate.
- **AES-256-GCM** — authenticated encryption for application traffic.
- **SHA-384** — hash function used by the suite and handshake construction.

ECDHE and Certificate Identity



Server Key Exchange: ECDHE using the X25519 elliptic curve.

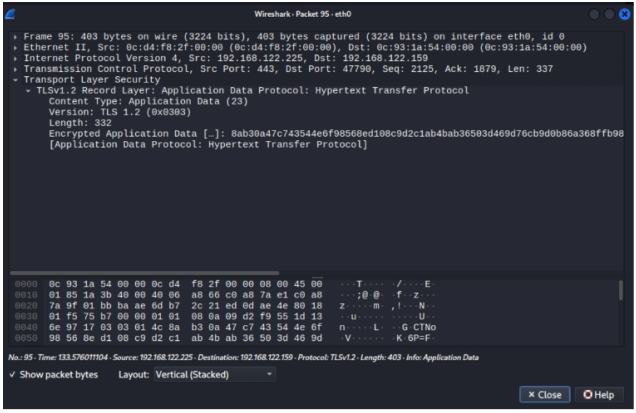


Certificate details show the same identity in Issuer and Subject for demo.local.

Security property

Ephemeral ECDHE creates temporary session keys and supports Perfect Forward Secrecy: compromise of the long-term RSA key does not automatically reveal previously captured session keys.

TLS 1.2: Handshake Completion

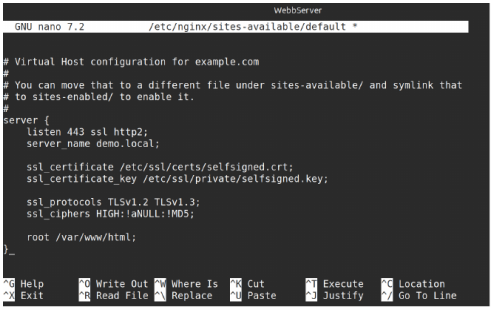


After the Finished messages, Wireshark shows encrypted TLS Application Data.

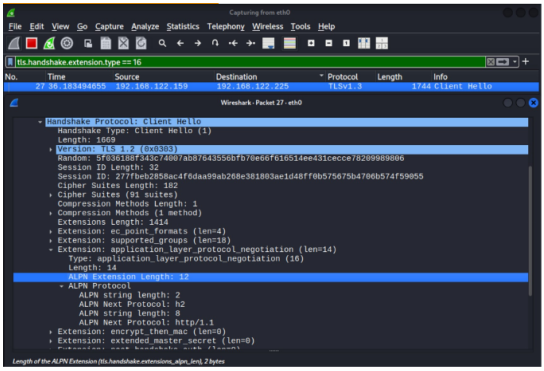
What this confirms

- Client and server completed the handshake.
- Shared traffic keys were established.
- HTTP content moved inside encrypted TLS records.
- The self-signed certificate still produced a trust warning at the client.

HTTP/2 Negotiation with ALPN



Nginx enabled HTTP/2 and allowed TLS 1.2 / TLS 1.3.



ClientHello ALPN extension advertises h2 and http/1.1.

Key observation

HTTP/2 is negotiated inside the TLS handshake through ALPN. The capture used TLS 1.3, so more of the handshake became encrypted earlier than in the TLS 1.2 capture.

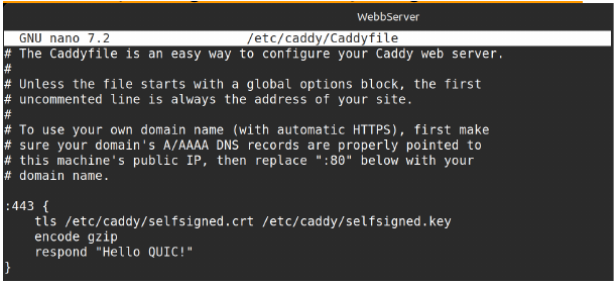
TLS 1.2, HTTP/2 and QUIC: Transport Comparison

Protocol	Transport	Handshake behaviour	Lab observation
TLS 1.2	TCP + TLS	TCP handshake, then a multi-flight TLS handshake	More visible handshake messages before Application Data
HTTP/2	TCP + TLS 1.3	TCP remains; ALPN selects h2 inside TLS	Earlier encrypted handshake traffic and fewer visible TLS messages
HTTP/3 / QUIC	UDP + QUIC + integrated TLS 1.3	Transport and cryptographic setup are combined	QUIC Initial / Handshake packets; first connection approximately 1-RTT

Important distinction

HTTP/2 improves multiplexing but still relies on TCP. QUIC removes TCP and integrates TLS 1.3 directly into the transport protocol.

HTTP/3 Server with Caddy v2

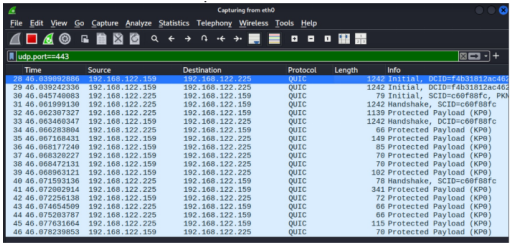


Caddy configured for port 443 with the lab certificate and a simple HTTP/3 response.

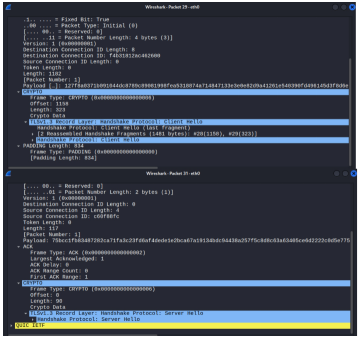
Permission problem Caddy initially could not read the private RSA key.

- Copied the certificate and key to `/etc/caddy/`.
- Changed ownership to the `caddy` service account.
- Restricted the private key with `chmod 600`.
- Verified the service was active after the fix.

QUIC and HTTP/3 Handshake



QUIC Initial, Handshake and Protected Payload packets over UDP/443.

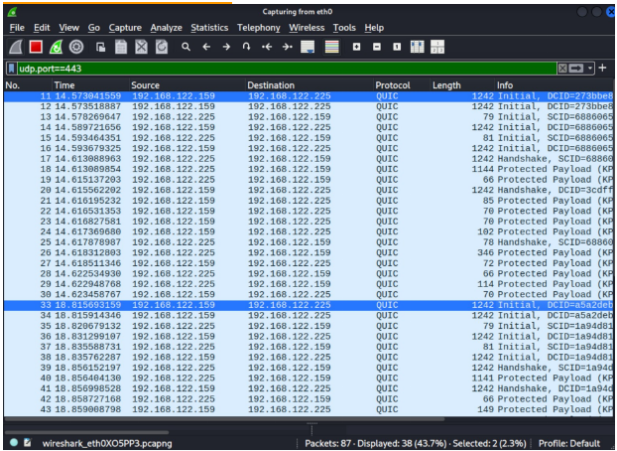


TLS 1.3 ClientHello and ServerHello carried inside QUIC CRYPTO frames.

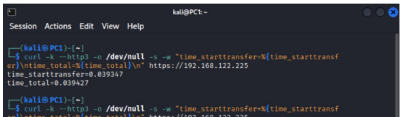
Key difference

TLS 1.3 is part of QUIC instead of running as a separate layer above TCP. This reduces connection setup latency and avoids TCP head-of-line blocking between independent QUIC streams.

0-RTT vs 1-RTT: What the Capture Showed



Two QUIC connections showed almost the same Initial / Handshake / Protected Payload pattern.

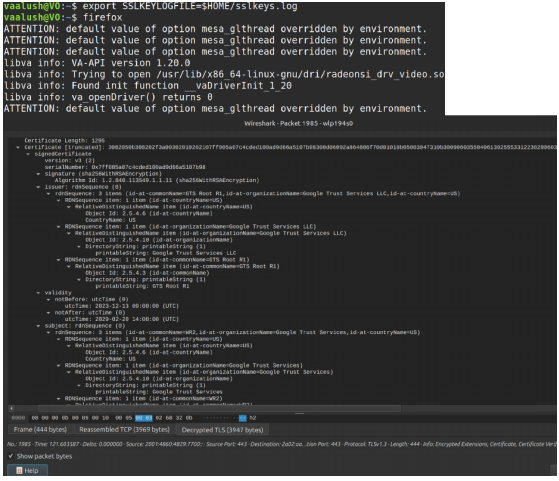


The second connection was slightly faster in curl timing.

Evidence limit

No clear application data was observed before the second handshake completed, so the lab did **not** claim that 0-RTT was successfully demonstrated.

Public Certificate Chain and TLS Decryption

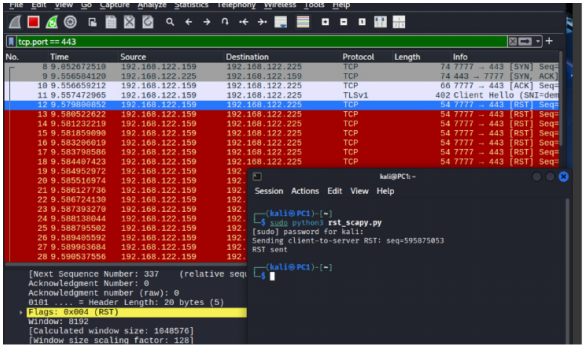


Firefox SSLKEYLOGFILE allowed Wireshark to decrypt the TLS 1.3 session and expose the certificate chain.

Trust comparison

- Self-signed lab certificate: Issuer = Subject.
- Public chain: server certificate signed by an Intermediate CA.
- Observed chain included WR2 and GTS Root R1.
- The browser trusts the chain because the root is already in the trusted certificate store.

TCP Reset Testing: From Failed Attempts to Success



A Scapy script generated RST packets, but invalid sequence numbers meant they were ignored.

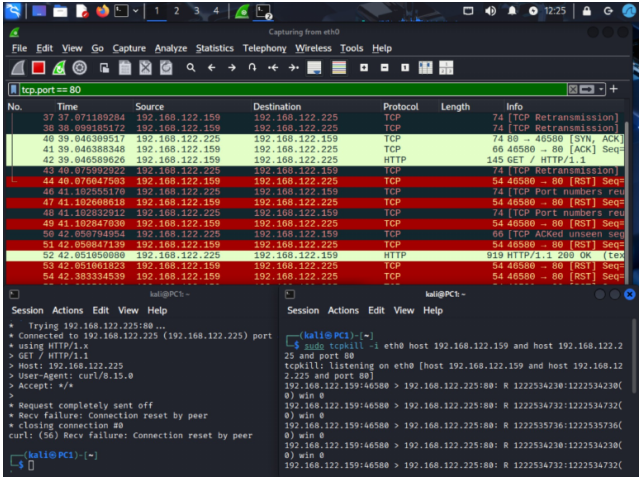
Three approaches

- **hping3**: handshake completed too quickly to obtain a usable sequence number.
- **Scapy**: packets were sent, but the sequence context was still wrong.
- **tcpkill**: watched the live TCP session and generated valid RST packets from observed traffic.

Lesson

TCP reset injection depends on matching the active 4-tuple and valid sequence space; simply setting the RST flag is not enough.

HTTP/1.1 Reset: Transport Layer Still Matters

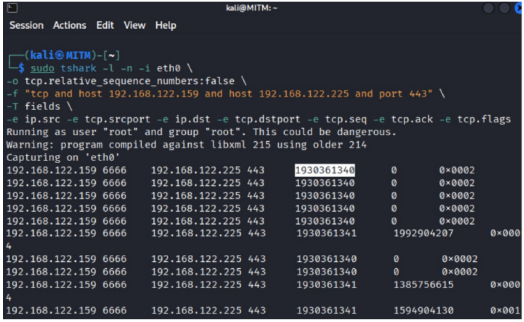


tcpkill reset an HTTP/1.1 connection after a small server-side delay created enough time for injection.

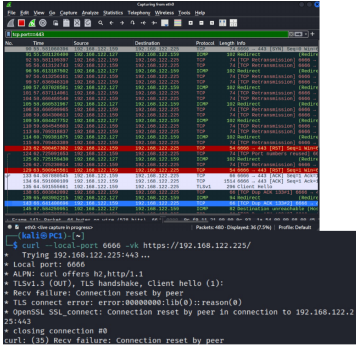
Observed result

- Valid RST arrived in the active TCP sequence space.
- The request was interrupted before the HTTP exchange completed.
- curl reported `Recv failure: Connection reset by peer`.
- Encryption protects content, but it does not prevent a valid transport-layer reset from terminating a TCP session.

MITM Position and Manual RST Injection



After ARP spoofing, the MITM node could observe sequence and ACK values with tshark.



A fixed curl source port, server delay and observed sequence number allowed a spoofed RST to interrupt the TLS connection.

Timing matters

When an automated reset arrived too late, the TLS handshake completed and the reset only terminated the already-established session.

Security and Production Considerations

Certificate trust

Self-signed certificates are useful in labs, but production clients need a trusted chain or centrally managed private PKI. Repeatedly ignoring warnings trains unsafe behaviour and increases MITM risk.

0-RTT

QUIC 0-RTT can reduce latency for resumed sessions, but early data can be replayed. Use it only for operations that are safe to repeat.

Transport attacks

TLS encrypts and authenticates application data, but a valid TCP reset can still terminate a TCP-based TLS or HTTP session. Availability is a separate property from confidentiality.

Protocol selection

HTTP/2 keeps TCP and gains multiplexing; HTTP/3 moves to QUIC/UDP and integrates TLS 1.3, changing both latency and failure behaviour.

Project Summary

Implemented

- TLS 1.2 HTTPS service with a self-signed RSA certificate.
- Wireshark analysis of ClientHello, ServerHello, ECDHE, certificate and Application Data.
- HTTP/2 negotiation through ALPN under TLS 1.3.
- Caddy HTTP/3 service and QUIC handshake analysis.
- Public certificate-chain inspection using exported TLS session keys.
- TCP reset tests from direct and MITM positions.

Evidence and lessons

- X25519 ECDHE provided ephemeral key agreement in the observed sessions.
- QUIC integrated TLS 1.3 directly into the transport handshake.
- No clear 0-RTT application data was observed, so the result remained inconclusive.
- Successful RST injection required valid live TCP sequence information.

Security result

The lab connects cryptographic trust, protocol performance and transport-layer availability by validating both normal handshakes and deliberately interrupted sessions.